Afin d'organiser le travail par étapes et de favoriser une progression régulière, quatre dépôts sont prévus : le 15 avril, le 30 avril, le 15 mai, puis un dépôt final le jour de la soutenance (à la dernière séance de l'AMS). Chaque dépôt doit être accompagné d'un Readme expliquant la progression depuis le précédent dépôt. Le Readme du premier dépôt résume en quelques lignes le travail fourni. Le correcteur veillera à ce que ces rendus témoignent d'une évolution continue tout au long du semestre. Cette progression constitue un critère essentiel dans l'évaluation.

Votre travail doit pouvoir :

- être compilé sur les ordinateurs du CERI en tapant la commande make,
- être exécuté sur les machines du CERI.

Dans un terminal :

```
$ mkdir Nom-Prenom

$ cp *.cpp *.h makefile Nom-Prenom/

$ tar cvfz Nom-Prenom.tgz Nom-Prenom/

Seuls les fichiers « .tgz » peuvent être déposés
```

En remplaçant bien sûr <Nom> par votre nom et <Prenom> par votre prénom !

Projet L1 Informatique
Dictionnaire intelligent et correcteur orthographique basé sur l'utilisation d'un arbre lexicographique

Fonctionnalités obligatoires

1. **Constructeur sans argument**
2. **Constructeur avec un nom de fichier comme argument**
3. **Destructeur**
4. **Constructeur par recopie**
5. **Insertion d'un mot dans l'arbre lexicographique**
6. **Recherche exacte d'un mot dans l'arbre lexicographique**
7. **Suppression d'un mot de l'arbre lexicographique**
8. **Comptage du nombre total de mots dans l'arbre lexicographique**
9. **Calcul de la longueur du mot le plus long**
10. **Une fonction de sauvegarde du dictionnaire contenu dans l'arbre lexicographique dans un fichier.**
11. **Voir plus loin**
12. **Voir plus loin**
13. **Voir plus loin**
14. **Voir plus loin**
15. **Voir plus loin**

Saisie de texte

11. Ecrire une fonction membre de la classe Arbre qui prend en argument un début de mot (préfixe) et qui renvoie tous les mots qui qui commencent par ce préfixe. Evidement, on suppose que le constructeur qui prend « un fichier dictionnaire » en argument a déjà été appelé c'est-à-dire que le dictionnaire est déjà représenté par l'Arbre lexicographique.

12. On souhaite développer une fonctionnalité permettant à l'utilisateur de saisir un texte avec une aide à la correction orthographique. Il s'agit d'écrire une fonction qui permet à l'utilisateur de saisir un texte, lequel sera enregistré dans un fichier dont le nom est fourni par l'utilisateur.

Lors de la saisie, l'utilisateur peut ne pas connaître l'orthographe exacte d'un mot. Dans ce cas, il saisit les premières lettres du mot, suivies d'une tabulation (ou d'un autre caractère déclencheur). Le

programme propose alors k mots commençant par ce préfixe. Si l'utilisateur ne trouve pas le mot recherché parmi les suggestions proposées, il peut appuyer de nouveau sur la touche de tabulation afin d'afficher les suggestions suivantes. S'il trouve le mot souhaité, il peut le sélectionner, puis continuer la saisie du texte. Si le mot n'existe pas dans le dictionnaire, l'utilisateur peut le saisir entièrement ; ce mot sera alors ajouté au dictionnaire. Après chaque interaction entre l'utilisateur et le programme, le texte complet doit être réaffiché.

Dans cette question, vous pouvez ranger le texte du fichier dans une liste chaînée (par exemple).

Correction orthographique

On a un dictionnaire de mots stocké dans un arbre à tête commune. Etant donné un mot w et un entier k, nous souhaitons obtenir les k mots de notre dictionnaire les plus proches de w, selon une distance de similarité que nous définissons ci-dessous.

<u>Distance de similarité</u>

Nous souhaitons établir une sorte de distance entre deux mots. Trois types d'erreurs sont possibles : substitution, insertion et suppression. Chacune de ces erreurs augmente la distance de 1 pt.

Exemple :
distance ("chien", "chen"), une suppression donc 1pt.
distance ("chen", "chien"), une insertion donc 1pt.
distance ("chuen", "chien"), une substitution donc 1pt.

Soient deux mots w et v de tailles respectivement n et m. Nous construisons une matrice D de taille $(m+1) \times (n+1)$.

Initialisation :
$D[0][j] = j$ (insérer j caractères) et $D[i][0] = i$ (supprimer i caractères)

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(suppression)} \\ D[i][j-1] + 1 & \text{(insertion)} \\ D[i-1][j-1] + \delta & \text{(substitution)} \end{cases}$$

Avec :

$$\delta = \begin{cases} 0 & \text{si } w_i = v_j \\ 1 & \text{sinon} \end{cases}$$

En réalité $D[i][j]$ représente la distance entre les i première lettres de w et les j premières lettres de v.
Voici un exemple avec $v =$ "chien" et $w =$ "shein".
On initialise $D[0][j] = j$ et $D[i][0] = i$.
Dans cet exemple $distance(w, v) = D[5][5] = 2$

		s	h	e	i	n
	0	1	2	3	4	5
c	1	1	2	3	4	5
h	2	2	1	2	3	4
i	3	3	2	2	2	3
e	4	4	3	2	3	3
n	5	5	4	3	3	2

Exercice :

13. Étant donnés deux mots calculer la distance entre les deux mots en utilisant l'algorithme ci-dessus. Testez votre fonction dans différents cas d'utilisation.

Revenons maintenant au problème que nous souhaitons résoudre. L'utilisateur soumis un texte à votre programme. Le programme doit trouver tous les mots hors vocabulaire. Pour chaque mot hors vocabulaire, le programme doit proposer une liste des k mots les plus proches selon la distance décrite ci-dessus.

Voici une stratégie possible : à chaque noeud de l'arbre lexicographique, on maintient une ligne de la matrice D.

$$D[j] = distance([w_0, w_1, \ldots w_j], le_prefixe_courant)$$

Initialisation:

À la racine (préfixe vide) : $D[j] = j$ pour j allant de 1 à m.

Descente dans l'arbre:

$$D'[0] = D[0] + 1$$

$$D'[j] = \min \begin{cases} D'[j-1] + 1 & \text{(insertion)} \\ D[j] + 1 & \text{(suppression)} \\ D[j-1] + \delta & \text{(substitution)} \end{cases}$$

Avec :

$$\delta = \begin{cases} 0 & \text{si } w_j = c \\ 1 & \text{sinon} \end{cases}$$

Que faire quand on trouve un mot:
À cet instant $D[m]$ correspond à la distance entre le mot trouvé et le mots pour lequel nous sommes en train chercher les plus proches.

À vous de gérer la façon de garder les k plus proches mots

Optimisation clé : élagage (pruning)
À chaque Noeud, on regarde : $min(D)$.
Si $min(D)$, la distance associée à un nœud donné est déjà trop élevée, on interrompt l'exploration, car toute poursuite ne pourra qu'augmenter cette distance.

14. Implémentez une méthode de la classe *Arbre* prenant en paramètre un mot (sous forme de chaîne de caractères) et retournant les k mots les plus proches, dont la distance à ce mot est inférieure à un seuil donné.

15. Écrire une fonction qui prend en argument un fichier texte. Pour chaque mot absent du dictionnaire, la fonction propose à l'utilisateur k mots suffisamment proches. L'utilisateur peut choisir l'un des mots proposés. Il peut également valider le mot d'origine ; dans ce cas, celui-ci est ajouté au dictionnaire. Le nouvelle version du fichier doit être stockée dans un autre fichier ayant le même que le fichier d'origine avec une extension « .cor »

Dans cette question, vous pouvez ranger le texte du fichier dans une liste chaînée (par exemple).

16. Proposer une version interactive simple utilisant le terminal